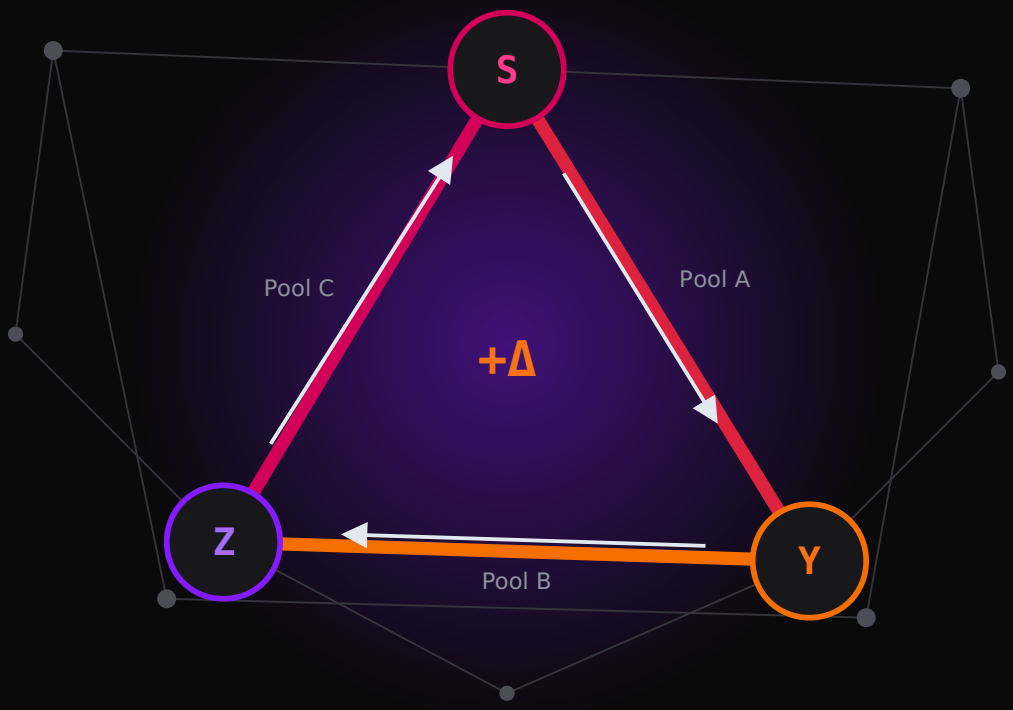
PULSECHAIN · ON-CHAIN ARBITRAGE SCANNER

Arb-Me-Now

Process Manual & White Paper

Methodology, operating procedure, verified parameters and risk model



PulseX v1 · v2 | 9mm v2 | LibertySwap V3 | 2-pool + triangular | BigInt engine | read-only

Version 4.9 · November 2026 · Analysis tool, not financial advice

cryptales.github.io/Arb-Me-Now

Contents

1. Purpose 3
2. Architecture 3
3. Methodology 4
4. Operating Procedure (step by step) 7
5. Interpreting results 8
6. Risks and limitations 9
7. Security model 9
8. Maintenance 10
9. Roadmap 10
10. Version history 10

At a glance. Arb-Me-Now reads every pool from one PulseChain block, models each swap with exact integer maths, finds the optimal size for 2-pool and triangular routes, and reports profit before and after estimated gas. It never connects a wallet or sends a transaction.

1. Purpose

Arb-Me-Now finds price differences between decentralised exchanges (DEXs) on PulseChain and tells you whether a round-trip trade would make money **after DEX fees**. It reads live pool data straight from the chain, so all numbers come from a single block. It never signs or sends transactions, and it never asks for a wallet or private key.

2. Architecture

Layer	What it does
Data	Public RPC `rpc.pulsechain.com`. All pool states are read in **one** Multicall3 `tryBlockAndAggregate` call (selector `0x399542e9`), so every price comes from the same block. PulseChain RPC rejects JSON-RPC batch requests, so Multicall3 is required.
Tick data (v4.5, adaptive since v4.8)	A second Multicall3 call, pinned to the **same block**, reads `ticks(int24)` (selector `0xf30dba93`) for an adaptive number of boundary ticks on each side of every v3 pool (see §3.9; cap 1–8, default 4), giving `liquidityNet` at each crossing. If this read fails, the engine falls back to single-interval maths.
Discovery	`getPair` / `getPool` lookups on each factory for a fixed list of liquid pairs over the watch list (v4.7: WPLS, PLSX, HEX, eHEX, INC, WETH, DAI and bridged stables; new pairs WPLS/eHEX, HEX/eHEX, eHEX/DAI, WPLS/INC, PLSX/INC, INC/HEX, WETH/DAI, WETH/PLSX, WETH/HEX).
Engine	Pure BigInt maths (`engine.js`) with exact integer swap formulas and no floating-point rounding in the money path. It is unit-tested in Node (`test.js`).
UI	A single self-contained `index.html` hosted on GitHub Pages, with no backend.
External	LibertySwap quote API (bridge checks) and Coinbase spot (ETH/USD reference only).

Venues and verified fees

Venue	Factory	Fee
PulseX v1	`0x1715a3E4A142d8b698131108995174F37aEBA10D`	0.29%
PulseX v2	`0x29eA7545DEf87022BAdc76323F373EA1e707C523`	0.29%
9mm v2	`0x3a0Fa7884dD93f3cd234bBE2A0958Ef04b05E13b`	0.25%
LibertySwap V3	`0x796fcbDC956b85797EFe21145Aa97599B7FB36a6`	0.01 / 0.05 / 0.25 / 1% tiers

All fees were checked on-chain, not taken from documentation.

3. Methodology

3.1 Swap maths

- **v2 (constant product):** `out = in·(10000−fee)·R_out / (R_in·10000 + in·(10000−fee))`, using integer division the same way the pool contract does.
- **v3 (concentrated liquidity, multi-tick since v4.5):** exact swap maths using `sqrtPriceX96` and active liquidity `L`, walking across up to **N tickSpacing intervals per direction** (N adaptive since v4.8, §3.9; cap 1–8, default 4). When the price crosses a boundary tick, `liquidityNet` is added (moving up) or subtracted (moving down) to get the next interval's `L`. The fee is taken on the gross input. Any trade that would go beyond the last interval read, or into an interval with zero liquidity, is treated as infeasible. If tick data is unavailable, the engine falls back to the conservative single-interval model.

3.2 Safe-depth caps

- A v2 leg may use at most **10% of the input reserve**.
- A v3 leg may not move the price beyond the **4th tick interval** read in its direction (or the current interval in single-interval fallback).

3.3 Route types

1. **Two-pool cycles:** S → Y on pool A, then Y → S on pool B, where A and B are different pools for the same pair.
2. **Triangular 3-hop routes (v4.4):** S → Y → Z → S across three different pools.

3.4 Optimal sizing

For each route, the engine first finds the largest feasible input size by binary search. It then runs a ternary search for the size with the highest profit, which works because profit rises and then falls as size grows. Finally it checks every size in the last narrow window exhaustively. It also evaluates the size you typed in, so you can compare your size with the best one.

3.5 Filtering and pricing

- **Suspicious/dust:** routes where any leg has under the dust threshold in USD depth, or an implausibly large price gap, are hidden by default.
- **USD values:** WPLS/USD comes from the deepest WPLS/stablecoin pool, treating bridged stables as $1. Other tokens are priced through their deepest WPLS pool.
- **Snapshot age:** data is marked stale after 20 seconds.

3.6 Bridge price check (LibertySwap)

`GET apis.libertyswap.finance/v3/swap/quote` gives the real quote from PulseChain to Ethereum (fee 0.5% for USDC and WETH from 369 → 1). The result is compared with the PulseChain purchase price and the Coinbase spot price. **This is not atomic:** the bridge takes minutes and you carry price risk while it runs. The API rate limit is 30 requests per 60 seconds and is enforced on the client.

3.7 Atomic execution: ArbExecutor (v4.6)

`contracts/src/ArbExecutor.sol` (Solidity 0.8.24, Foundry) runs a 2- to 4-leg closed cycle S → … → S in **one transaction**: - **Flash swap on leg 0:** the first pool sends its output first; the remaining legs run inside its callback; leg 0 is repaid in S at the end. No starting capital is needed. - **All-or-nothing:** if the cycle returns less than the amount owed it reverts `NoProfit(returned, required)`; it also reverts if the leftover is below `minProfit`. Only gas can be lost. Profit is sent to the owner. - **Callbacks (verified in on-chain bytecode):** PulseX v1/v2 `uniswapV2Call`, 9mm v2 `pancakeCall`, LibertySwap V3 `libertyV3SwapCallback` (`0x31e2ba55`). - **Safety:** `execute` by the owner or the optional keeper (v4.9, §3.11); `withdraw` / `setKeeper` owner-only; callbacks accepted only from the pool currently being swapped
while a run is active; deadline; token-continuity check; fee-on-transfer tokens unsupported. - **Leg encoding (uint256):** bits 0–159 pool · 160–167 kind (0 v2, 1 v3) · 168–175 zeroForOne · 176–191 v2 fee bps. Call: `execute(uint256[] legs, uint256 amountIn, uint256 minProfit, uint256 deadline)` (selector `0x29df2c6f`). - **Testing:** 11/11 Foundry tests on a PulseChain fork (live route reverts NoProfit; induced gaps profitable for v2→v2, v3-flash→v2, v2→v3 and a 3-hop; access control, stray callbacks, expiry, open cycles). A JS end-to-end test on an anvil fork drove the scanner's own encoder: contract profit matched the engine's prediction **exactly** for v2→v2 and within **2 wei** with a v3 leg, and a real execute tx paid the owner. Measured `execute` gas (Foundry report): median ≈ 294k, max ≈ 612k.

3.8 Wallet send (v4.7)

Trade Companion's opt-in **Connect & Send** uses the injected EIP-1193 wallet (MetaMask): `eth_requestAccounts` → `wallet_switchEthereumChain` to `0x171` (369) → `owner()` read on the executor; sending is blocked unless the connected account is the owner → `eth_sendTransaction` with the built `execute(...)` calldata → the tx hash is shown with an explorer link. Keys never leave the wallet.

3.9 Adaptive tick depth (v4.8)

For each V3 pool the engine first computes the current interval's capacity C in start-token (S) units: the smaller of `v3MaxIn` with S as input and `v3MaxIn` with the other token as input, converted to S at the pool's sqrtPrice. It then reads `d = clamp(⌊size / C⌋ + 2, 2, cap)` tick intervals per direction, where cap is the user's slider value (1–8; cap 1 means single-interval maths). Pools that do not contain S (middle legs of 3-hop routes) read the full cap. Live check (2026-11-01, WPLS start): at 1,000 WPLS deep pools read 2 intervals; at 10^8 WPLS they read 6–8; with cap 1 every pool reads 1.

3.10 Piteas cross-check and the Tide venue (v4.8.1)

Piteas (piteas.io) is a DEX aggregator, not a DEX: it holds no liquidity and routes swaps across PulseChain venues through its router `0x6BF2…88A6`. Trade Companion's opt-in **Check with Piteas** button asks `GET https://sdk.piteas.io/quote` for the cycle's first hop (S → first intermediate token, at your input size) and then the way back (intermediate → S), i.e. 2 requests. It shows both outputs, the venues used, and the Piteas round trip against your input and against our route's output: - **Piteas round trip ≥ our route:** the gap is likely available market-wide, so other traders can take it too. - **Our route beats it:** the gap looks specific to the pools we found.

Piteas allows **10 requests per minute** (going over can block the IP for an hour) and states the API is not meant for arbitrage bots. So the check is manual only, capped locally at 6 requests per rolling 60 s, and never part of the automatic scan. The scanner does not depend on Piteas; if the API is down (it

returned HTTP 500 for a period on 3 Nov 2026), the panel shows "Piteas unavailable" and everything else keeps working. Piteas quotes cannot be an ArbExecutor leg: they have no flash callback, and their calldata is built off-chain for one exact amount.

Tide appeared in a Piteas route. On-chain inspection (3 Nov 2026) shows the pool `0x6926…E38B` ("RISKOFF") is a **Balancer V3-style weighted pool**, not a Uniswap-style pair: it exposes `getVault()` (vault `0x634f…4b2d`), `onSwap`, `getWeightedPoolImmutableData` and `getCurrentLiveBalances`, and reverts on `getReserves()`. It holds 5 tokens (HEX, INC, CST, PLSX, WPLS) at weights 5/5/80/5/5 %, with a 0.30 % swap fee. Its live balances were small (≈14.8k HEX, 74 INC, 61k CST, 4.49M PLSX, 3.26M WPLS). Adding it would need weighted-pool maths in the engine **and** a Balancer-V3 vault integration in ArbExecutor, for very little depth, so it is **not scanned** for now. The Piteas panel labels Tide hops "(not scanned here)".

3.11 Deploy kit and keeper role (v4.9)

Keeper role. The `owner` is fixed at deployment (immutable, no transfer). Since v4.9 the owner can appoint one optional **keeper**, a separate bot key, with `setKeeper(address)`; `setKeeper(0x0)` revokes it. - The keeper may call `execute()` only. `withdraw` and `setKeeper` revert `NotOwner` for it. - Profit is always transferred to the **owner**, never to the caller. - Unauthorised callers get `NotAuthorized` (`0xea8e4eb5`); the zero address can never act as keeper. - So a leaked keeper key can at worst waste its own gas on reverting trades. It cannot move funds. - Fork tests (14/14 pass, 3 new): keeper executes a profitable cycle and the owner is paid while the keeper's balance is unchanged; keeper cannot withdraw or set keeper; a revoked keeper is rejected.

Deploy kit. `foundry.toml` pins solc 0.8.24, EVM `paris` (no newer opcodes, for maximum PulseChain compatibility), optimizer 200. The build embeds that exact creation bytecode (5,753 bytes, keccak `0x1635…1004`) in the page. In Trade Companion, after **Connect Wallet**: 1. **Estimate deploy cost:** `eth_estimateGas` of the creation tx from your address plus `eth_gasPrice`; nothing is sent. On 3 Nov 2026: about 1.56M gas at ~320k gwei, roughly 500 PLS. 2. **Deploy ArbExecutor:** a confirm dialog warns that the connected account becomes the permanent owner. The wallet signs the creation tx (gas = estimate + 20%). The page waits for the receipt, reads `owner()` back, and saves the address in this browser. 3. **Set / revoke keeper:** owner-checked before sending; the owner's own address is refused as keeper.

End-to-end fork test (3 Nov 2026, local anvil fork of PulseChain at block ~27.97M, injected test wallet; nothing touched mainnet). - Estimate: 1,297,519 gas, about 367 PLS. - Deploy from the owner address succeeded, and `owner()` matched. - Read keeper showed none, then Set keeper worked, and Dry run said "Would succeed, +52.86 WPLS". - The **keeper** sent the 3-leg route (7,122 WPLS flash input). Status 1: the owner received **52.86 WPLS** and the keeper received 0. - A stranger's Send and Set keeper were blocked by the page; Revoke worked. - **Important:** that execute used **770,688 gas**, about 218 PLS at 282,679 gwei. That is far more than the 52.86 WPLS profit (WPLS = PLS 1:1), so live it would have **lost about 165 PLS net**. It is also about 2.2× the scanner's 350k 3-hop gas assumption, so "net after gas" for routes with V3 legs was badly optimistic. **Fixed in v4.9.1** (§3.12).

Connect & Send now also accepts the keeper account. **Dry run** still simulates from the owner. The CLI alternative is `forge create … --interactive` on your own machine.

3.12 Gas model recalibration (v4.9.1)

The old flat estimates (250,000 gas for 2 hops, 350,000 for 3) came from generic DEX swaps. They did not include the ArbExecutor's flash-swap callbacks or the extra cost of V3 swaps. v4.9.1 replaces them with a per-route model based on measured gas:

Measured on a PulseChain fork (3 Nov 2026). The forge rows are the `execute` call only, with pools already warm from test setup, so they understate a real transaction.

Route (legs)	Measured gas
V2 flash + V2	383,256
V3 flash + V2	428,225
V2 flash + V3 (large size, tick crossings)	573,416
V2 + V2 + V2 triangular	346,153
3-leg route with V3 legs (full transaction, cold)	770,688

Model: estimated gas = **150,000 base** (transaction + flash/callback overhead) + **140,000 per V2 hop** + **230,000 per V3 hop**. Values are rounded up to allow for cold storage access.

Route	Old estimate	New estimate
V2 + V2	250,000	430,000
V3 + V2	250,000	520,000
V2 × 3	350,000	570,000
V3 + V3 + V2	350,000	750,000
V3 × 3	350,000	840,000

- Each row's "net after gas" uses its own estimate and shows it, e.g. "(~520k gas)". The summary line states the model.
- **Dry run now measures real gas.** After a successful `eth_call`, the page also calls `eth_estimateGas` for the exact transaction from the owner and shows its gas, its cost in PLS/USD at the live gas price and, for WPLS routes, **net profit after gas**. That figure overrides the model.
- The model is still an estimate. Tick crossings, token transfer logic and storage warmth change real gas by tens of thousands. Always rely on the dry-run number before sending.

4. Operating Procedure (step by step)

1. Open https://cryptales.github.io/Arb-Me-Now/.
2. Under **Settings**, choose the start token and the trade size, and turn venues on or off.
    - **Min profit (bps of input):** the smallest return you care about, in basis points of the amount you put in (1 bp = 0.01%; 10 bps = 0.1%; 100 bps = 1%). It is checked against the profit at the **optimal size**, before gas. It only takes effect when **Only rows ≥ min profit** is ticked;

otherwise every row is shown. The default is 0, so any profitable route passes. Example: 1,000 WPLS in, 1,002 WPLS out = 2 WPLS profit = 20 bps. The LibertySwap bridge quotes and Triangular 3-hop routes toggles are on by default.

3. Click **Scan**. Check the telemetry bar: RPC is **OK**, the snapshot block is shown, and the snapshot age is under 20 s.
4. Read the results table, which is sorted by profit in USD. Each row shows the route, venues, mid-price gap, best size, profit at the best size, profit at your size, and flags.
5. Click **Inspect** to open the Trade Companion. It shows reserves or v3 state for every leg, the depth caps, and the exact BigInt calculation.
6. **Before acting on any row:**
   - Re-scan to confirm the opportunity is still there in a fresh block.
   - Check the **net after gas** line. Gas is estimated from the live `eth_gasPrice` × a per-route estimate (150k base + 140k per V2 hop + 230k per V3 hop, §3.12), valued at WPLS/USD. The Trade Companion **Dry run** shows the measured `eth_estimateGas` figure and net after gas. Use that number before sending.
   - Allow for MEV and front-running risk. A public mempool trade can be sandwiched.
   - Check the pool addresses on scan.pulsechain.com.
7. **Alerts (v4.5):** tick 🔔 **Alert on new profitable route** and allow browser notifications when asked. Pair it with **Auto-refresh every 15 s**. The scanner beeps, shows a banner and sends a notification **once** for each newly profitable route that is ≥ your min bps and still positive after gas (suspicious rows never alert). Alerts only work while the tab stays open.
8. **Execute (v4.6, optional):** deploy your own ArbExecutor once, either with 🚀 **Deploy ArbExecutor** in Trade Companion (v4.9; signed in your wallet) or with `forge create src/ArbExecutor.sol:ArbExecutor --rpc-url https://rpc.pulsechain.com --interactive --broadcast` on your own machine. Optionally set a keeper (§3.11). In Trade Companion, enter its address, keep or edit size and min profit (default 50% of expected), click **Build calldata**, then **Dry run** (an `eth_call` from the owner; nothing is signed). If it says "Would succeed", send the calldata from your owner wallet (value 0) before the 120 s deadline.
9. **Piteas cross-check (v4.8.1, optional):** in Trade Companion, click **Check with Piteas** to compare the route with the aggregator's whole-market round trip (2 requests; max 6 per minute). Use it sparingly.
10. For bridge checks, treat the edge as an upper bound. Gas on both chains, bridge time and slippage reduce it.

5. Interpreting results

Display	Meaning
`not profitable at any size`	Fees are larger than the price gap at every size.
`+x TOKEN` profit	Gross profit at the optimal size, before gas.
`net after gas`	Profit in USD minus the estimated gas cost (green = still profitable).
`v3 multi-tick (≤N intervals)`	v3 legs are sized across up to N tick intervals; liquidity beyond that is not modelled.

Display	Meaning
`v3 in-interval (conservative cap)`	Fallback when tick data could not be read: size limited to the current tick.
✅ `Would succeed` / ❌ `Would revert NoProfit`	ArbExecutor dry-run result at the latest block.
🔔 banner / notification	A new profitable, net-of-gas route ≥ min bps appeared (deduplicated per route).
`3-hop triangular`	Triangular route. Only profitable ones are listed.
Hidden: suspicious/dust	Thin or anomalous pools have been left out for your safety.

Live findings (1 Nov 2026, block ~27.94M): no profitable two-pool or triangular routes after DEX fees. The best gross gap was 0.22% (WPLS–WETH, PulseX v2 → LibertySwap V3 0.25%), which is below the combined 0.54% in fees. At about 444,000 gwei, the estimated gas is under $0.001 per route, so DEX fees, not gas, are what close the gap. The WETH bridge check showed about +0.17% to +0.30% against Coinbase before gas and slippage. Efficient markets tend to close these gaps fast, so the tool is most useful during volatility.

6. Risks and limitations

- Gas is an estimate (fixed gas units × live gas price). MEV and pending-block state are not modelled.
- v3 depth is limited to the adaptive tick depth (max 8 intervals per direction); liquidity beyond that is not modelled.
- ArbExecutor: a public-mempool tx can be front-run or the price can move after the dry run; then it reverts and gas is lost. The contract is unaudited; use small sizes first.
- Alerts run in the browser only: the tab must stay open, and the browser or OS may block notifications or sound.
- The results are a snapshot. By the time a transaction lands, reserves may have changed.
- Bridge round-trips are non-atomic and carry price risk.
- The public RPC can lag or rate-limit you.
- **This is analysis, not financial advice. Always verify on-chain before trading.**

7. Security model

By default the page is read-only: it uses only `eth_call` and HTTPS GET requests (PulseChain RPC, LibertySwap API, and, only when you click the button, the Piteas quote API). Since v4.7 there is an **opt-in** wallet connection: a transaction is sent only when you click Connect & Send and approve it in your wallet, and only from the executor owner. The page never sees or stores private keys; there is no tracking and no backend. All code is in one auditable HTML file.

8. Maintenance

- **Source files:** `engine.js` (engine), `ui.js` (UI logic), `body.html` (markup) and `test.js` (Node tests: `node test.js`).
- **Build:** the CSS + `body.html` + `engine.js` (without module exports) + `ui.js` are combined into one `index.html`.
- **Deploy:** commit `index.html` to `main`. GitHub Pages serves it within about a minute.
- **When fees or factories change:** re-verify on-chain and update the constants in `engine.js`.

9. Roadmap

first mainnet deploy + small live test · v4.10 private/bundled submission (MEV protection) · v5.0 server-side / Telegram alerts · Balancer-V3 / Tide support if its liquidity grows · more venues and tokens.

10. Version history

- **v4.9.1:** Gas model recalibrated from fork measurements: per-route 150k base + 140k per V2 hop + 230k per V3 hop (was flat 250k/350k, ~2.2× too low for V3 routes). Dry run now shows measured `eth_estimateGas`, PLS/USD cost and net-after-gas.
- **v4.9:** ArbExecutor keeper role (execute-only bot key; owner-only withdraw/setKeeper; profit to owner; 3 new fork tests, 14/14). Deploy kit: pinned paris build, in-app Estimate / Deploy / Set-keeper via the connected wallet; Send accepts the keeper.
- **v4.8.1:** Opt-in Piteas aggregator cross-check in Trade Companion (2 quotes, local 6/min guard, graceful "unavailable" handling); Tide venue identified on-chain as a Balancer-V3 weighted pool and documented (not scanned).
- **v4.8:** Adaptive V3 tick depth: per-pool interval count sized to the trade (1–8 cap slider, same-block reads), depth range in telemetry.
- **v4.7:** New tokens eHEX, INC (and WETH pairs) with 9 liquid pairs; opt-in MetaMask Connect & Send (chain switch, owner check, wallet-signed tx).
- **v4.6.1:** Fixed stale browser tab title (v4.3 → v4.6); white paper PDF regenerated.
- **v4.6:** ArbExecutor atomic flash-swap contract (PulseX v1/v2, 9mm, LibertySwap V3; 2–4 legs), fork-tested; Trade Companion execution panel with calldata builder and read-only dry run.
- **v4.5:** multi-tick LibertySwap V3 depth (up to 4 intervals per direction, same-block tick reads) and profitability alerts (sound + browser notification, deduplicated).
- **v4.4:** triangular 3-hop routes, gas-adjusted net profit, separate route counts in the summary, and old pages redirected to the scanner.
- **v4.3:** live Multicall3 reserves, verified fees, LibertySwap V3 and bridge quotes, and a BigInt engine.